NMIMS Indore

School of Technology, Management & Engineering

Project Report

Recommendation Systems

Implementing Collaborative Topic Regression (CTR)

Based on the Paper:

“Collaborative Topic Modeling for Recommending Scientific Articles”

Chong Wang & David M. Blei — KDD 2011

Submitted by

Simar Singh Khanuja

B.Tech — NMIMS Indore

Academic Year 2025–26

Table of Contents

Table of Contents 2
1. Abstract 3
2. Introduction 3
3. Literature Review & Related Work 4
3.1 Collaborative Filtering 4
3.2 Topic Models 4
3.3 CTR — The Paper I Replicated 4
3.4 Explainability 4
4. Proposed Methodology 5
4.1 Dataset Selection & Description 5
4.2 Feature Engineering 5
4.3 Data Splitting Strategy 6
4.4 Local Model Framework (LMF) 6
4.5 Global Model Framework (GMF) — CTR 7
5. Implementation Details 7
5.1 Step 1: Data Loading 7
5.2 Step 2: Feature Engineering 7
5.3 Step 3: Data Splitting 7
5.4 Step 4: Model Training 8
5.5 Step 5: Validation Evaluation 8
5.6 Step 6: Final Test Evaluation 8
5.7 Step 7: Hybrid Recommender / Analysis 8
6. Results & Analysis 9
6.1 Local Model Performance — SVD + Ridge Explainability 9
6.2 Global Model Performance — Recall@M 10
6.3 Paper vs. My Results 10
6.4 Ranking Metrics 11
7. Visualisations 11
7.1 MAE: Original SVD vs Shadow Model 11
7.2 Distribution of SVD Latent Factor 0 12
7.3 Original SVD vs Shadow Model Predictions 12
7.4 Top 10 Genres by Importance 13
7.5 Distribution of User Ratings 13

7.6 Ridge Coefficient Heatmap 14
7.7 LDA Topic Proportions 14
7.8 In-Matrix Recall@M 15
7.9 Out-of-Matrix Recall@M (Cold-Start) 15
7.10 CTR Offset Analysis 16
8. Conclusion 17
9. Future Work 17
10. References 18

1. Abstract

In this project, I implemented the Collaborative Topic Regression (CTR) model proposed by Wang and Blei in their 2011 KDD paper. The idea behind this paper is to combine two popular recommendation approaches — collaborative filtering (which looks at user behaviour) and content-based filtering (which looks at item features) — into a single unified model.

I applied this to the MovieLens 20M dataset, which has 20 million movie ratings. For the content part, I used genome tag scores (1,128 different tags per movie) instead of the paper's original bag-of-words from academic abstracts. I trained three models: SVD, PMF, LDA, and CTR, and evaluated them using Recall@M — the same metric used in the paper.

The main finding from my experiments is that genre features alone can explain only about 5.8% of what SVD learns from user behaviour, which confirms why CTR is needed. PMF and CTR both achieved around 0.79 Recall@300 in-matrix, while LDA alone only reached 0.15 — showing that collaborative signals are much stronger than content signals on this dataset.

2. Introduction

When I think about recommendation systems, the most common example that comes to mind is Netflix suggesting movies or Amazon recommending products. These systems generally work in one of two ways:

- Collaborative Filtering: looks at what similar users liked. For example, if you and another user both liked the same 10 movies, the system assumes you will also like the next movie they watched.
- Content-Based Filtering: looks at the features of the item itself. For example, if you liked an action movie, recommend more action movies.

Both of these have problems. Collaborative filtering fails completely when a new movie comes out and has zero ratings — this is called the cold-start problem. Content-based filtering ignores the actual preferences of users and treats everyone the same based only on item features.

The paper I am replicating, by Wang and Blei (KDD 2011), solves this by building a model called CTR (Collaborative Topic Regression) that combines both approaches. The key idea is that each item's latent vector in the recommendation model is tied to its topic proportions from an LDA topic model. So the model learns from both the content of items and the behaviour of users at the same time.

In my project, I applied this idea to movies instead of scientific articles. I used the MovieLens 20M dataset and implemented the full pipeline: SVD for the explainability part, and PMF + LDA + CTR for the main evaluation part.

3. Literature Review & Related Work

3.1 Collaborative Filtering

Collaborative filtering has been around since the 1990s. The most well-known modern version is SVD (Singular Value Decomposition), which became famous after the Netflix Prize competition in 2006-2009. Koren et al. (2009) showed that matrix factorisation beats older neighbourhood methods by learning a low-dimensional representation of users and items. The basic SVD prediction formula is:

```
predicted_rating = global_mean + user_bias + item_bias + user_vector · item_vector
```

For implicit feedback (like binary liked/not-liked), Hu et al. (2008) proposed confidence-weighted ALS (Alternating Least Squares), which is what I used in my PMF implementation.

3.2 Topic Models

LDA (Latent Dirichlet Allocation) was proposed by Blei et al. (2003) and is one of the most popular topic models. The idea is that each document is a mixture of topics, and each topic is a distribution over words. For movies, I used genome tag scores instead of words. LDA gives each movie a topic proportion vector θ_j which represents what 'topics' (themes/styles) the movie belongs to.

3.3 CTR — The Paper I Replicated

Wang and Blei (KDD 2011) combined PMF and LDA into one model. Instead of learning item vectors freely, CTR forces the item vector to be close to its LDA topic proportions:

```
item_vector = topic_proportions + offset

v_j = θ_j + ε_j
```

The ε_j is the collaborative offset — it captures what users tell us beyond what the content says. For brand new items with no ratings, the offset is set to zero and the prediction is just based on content. This elegantly solves the cold-start problem.

3.4 Explainability

For the first part of my project, I also trained a Ridge regression model to explain what SVD learned. The idea is simple: can we predict the SVD latent factors just from genre features? If yes, then genres explain the collaborative signals. If no (low R^2), then collaborative filtering is learning something much richer than genre labels.

4. Proposed Methodology

4.1 Dataset Selection & Description

I used the MovieLens 20M dataset because it is one of the standard benchmark datasets for recommendation systems research, and importantly it has a genome-scores.csv file with 1,128 tag relevance scores per movie. These tag scores are the closest equivalent to the bag-of-words features used in the original paper.

What	Details
Total ratings	20,000,263 ratings
Users (for CTR)	5,000 (subsampled for speed)
Movies (for CTR)	3,000 (subsampled for speed)
Rating scale	0.5 to 5.0 stars
What counts as 'liked'	Rating ≥ 4.0 (positive interaction)
Content features	1,128 genome tag relevance scores per movie
Genre labels (Part A)	19 binary genre flags (Action, Drama, etc.)

I subsampled to 5,000 users and 3,000 items for the CTR pipeline because the coordinate ascent algorithm is O(n_users × n_items) per epoch and would take too long on the full dataset on a laptop.

4.2 Feature Engineering

For Part A (SVD Explainability):

- Dropped the timestamp column since we don't need it.
- Split genre strings like 'Action|Comedy' into binary columns (one per genre).
- After training SVD, extracted its internal item factor matrix (called qi) as a DataFrame indexed by movie ID.
- Merged genre features with these latent factors so I could train Ridge regression on them.

For Part B (CTR Pipeline):

- Binarised ratings: 1 if rating ≥ 4.0, 0 otherwise.
- Loaded genome-scores.csv and pivoted it to a (movies × tags) matrix.
- Normalised each row to sum to 1.0 so it works as input to LDA.
- Removed movies with fewer than 5 ratings and users with fewer than 5 rated movies.

4.3 Data Splitting Strategy

I used two different splits depending on what I was testing:

In-matrix split (for testing warm items):

- 80% of each user's ratings go to training, 20% to test.
- When ranking recommendations, training items are hidden so the model can't cheat.

Out-of-matrix split (for cold-start testing):

- 10% of all movies are completely hidden from training.
- These movies have zero ratings in training — the model has never seen them.
- CTR predicts for these using: score = user_vector · topic_proportions (no offset since no ratings exist).
- PMF simply cannot do this — it has no vector for movies it has never seen.

4.4 Local Model Framework (LMF)

I trained three individual models:

Model 1: SVD (using Surprise library)

Standard matrix factorisation trained with stochastic gradient descent. Parameters: 20 latent factors, 20 epochs, learning rate 0.005, regularisation 0.02. This was used for the explainability experiment in Part A.

Model 2: PMF (my own implementation)

Probabilistic Matrix Factorisation using confidence-weighted ALS. Each rating ≥ 4.0 gets confidence weight a=1.0, and unobserved interactions get weight b=0.01. This is the collaborative filtering baseline — it uses no content features at all.

```
User update: u_i = (V^T C_i V + λ I)^-1 V^T C_i r_i
Item update: v_j = (U^T C_j U + λ I)^-1 U^T C_j r_j
```

Model 3: LDA (sklearn)

Fitted on genome tag scores to produce topic proportions for each movie. I used K=50 topics. User profiles are built by averaging the topic vectors of movies they rated positively. This is the content-only baseline.

4.5 Global Model Framework (GMF) — CTR

CTR is the main model from the paper. It modifies PMF by adding a content anchor to the item update:

```
Item vector = Topic proportions + Collaborative offset
v_j = θ_j + ε_j
```

The coordinate ascent updates from the paper (Equations 9–11) are:

```
u_i ← (Σ_j c_ij * v_j * v_j^T + λ_u * I)^-1 * Σ_j c_ij * r_ij * v_j
v_j ← (Σ_i c_ij * u_i * u_i^T + λ_v * I)^-1 * (Σ_i c_ij * r_ij * u_i + λ_v *
θ_j)
```

The important part is the λ_v * θ_j term at the end of the item update. This is what pulls item vectors towards their LDA topic proportions. Without it, CTR becomes identical to PMF.

I warm-started CTR from the PMF solution so it converges faster. For cold-start prediction, the formula simplifies to: score = user_vector · topic_proportions (since the offset ε_j = 0 for movies never seen in training).

5. Implementation Details

I implemented everything in a single Jupyter notebook (RSM_Project_Complete.ipynb) so it's easy to run. All the model code is written from scratch using NumPy and SciPy — I did not use any pre-built CTR library.

5.1 Step 1: Data Loading

- Loaded ratings.csv using pandas and kept only userId, movieId, and rating columns.
- Loaded genome-scores.csv and reshaped it from long format to wide format (movies as rows, tags as columns).
- Only kept movies that appear in both the ratings file and the genome file.
- Removed users and movies with fewer than 5 interactions each (iteratively until stable).
- For CTR: limited to 5,000 users and 3,000 movies for speed.

5.2 Step 2: Feature Engineering

- Normalised genome scores so each movie's row sums to 1.0.
- One-hot encoded genres from the pipe-separated string format.
- Extracted the qi matrix from SVD (item latent factors indexed by movieId).

5.3 Step 3: Data Splitting

- Part A (SVD): used Surprise library's train_test_split with 80/20 ratio and random seed 42.
- Part B (CTR): used sklearn's train_test_split per user with same ratio and seed.
- Cold-start: used numpy random generator with seed 42 to select 10% of movies as cold items.

5.4 Step 4: Model Training

Model	Key Settings
SVD	20 factors, 20 epochs, lr=0.005, reg=0.02, random_state=42
Ridge	alpha=1.0 (L2 penalty to avoid overfitting on correlated genres)
PMF	K=50, $\lambda_u=\lambda_v=0.01$, confidence a=1.0, b=0.01, 20 epochs
LDA	50 topics, 20 variational EM iterations, batch learning method
CTR	K=50, same λ and confidence as PMF, 20 epochs, warm-started from PMF

5.5 Step 5: Validation Evaluation

- MAE (Mean Absolute Error) computed on the Surprise test set for SVD and shadow model comparison.
- R^2 computed using sklearn's r2_score to measure how well genres explain SVD latent factors.
- Genre importance = average of absolute Ridge coefficients across all 20 latent dimensions.

5.6 Step 6: Final Test Evaluation

- Recall@M computed for M = 20, 50, 100, 200, and 300.
- For in-matrix: movies already seen in training are excluded from the ranking list.
- Only users who have at least one movie in the test set contribute to the average.
- For cold-start: the cold movies were never in training, so no exclusion needed.

5.7 Step 7: Hybrid Recommender / Analysis

- Computed the collaborative offset: $\varepsilon_j = v_j - \theta_j$ for every item in CTR.
- Compared the norms $||\varepsilon_j||$ and $||\theta_j||$ to see how much CTR deviates from the content prior.
- Saved all results to a JSON file for reproducibility.

6. Results & Analysis

6.1 Local Model Performance — SVD + Ridge Explainability

The first part of my experiment was about understanding how much of the SVD collaborative signal can be explained by genre labels.

Metric	Value
Training set size	16,000,210 ratings
Test set size	4,000,053 ratings
Original SVD MAE	0.6009
Shadow Model MAE (genres only)	0.6709
MAE increase	**+0.0700 (+11.6%)**
R^2 score	**0.0581 (genres explain only 5.81% of SVD latent space)**
Movies with learned SVD factors	25,857
Most important genre	IMAX (importance score: 0.070)

What this tells me: Even though genres are a simple way to describe movies, they can only explain about 6% of what SVD actually learned from 20 million user ratings. The remaining 94% of the collaborative signal comes from patterns in user behaviour that no genre label can capture. This is exactly why CTR is useful — its content prior θ_j starts from what we know about the movie, but the collaborative offset ε_j fills in everything the content misses.

The reason IMAX ranks as the most important genre is that IMAX movies form a very specific and narrow category (mostly large-scale blockbusters with unique viewing experiences). Drama, despite being the most common genre, ranks 9th because almost every movie has it — so it doesn't help distinguish movies from each other.

6.2 Global Model Performance — Recall@M

In-Matrix Results (warm items):

Model	R@20	R@50	R@100	R@200	R@300
PMF	0.2893	0.4614	0.6003	0.7280	0.7899
LDA	0.0112	0.0221	0.0634	0.1108	0.1521
CTR	0.2893	0.4614	0.6003	0.7280	0.7899

Out-of-Matrix Results (cold-start):

Model	R@20	R@50	R@100	R@200	R@300
LDA (cold)	0.0000	0.0000	0.0000	0.0000	0.0000
CTR (cold)	0.0000	0.0000	0.0000	0.0000	0.0000
PMF	N/A	N/A	N/A	N/A	N/A

Why CTR equals PMF in-matrix: When I looked at the LDA output (Plot 7), I noticed that all movies got nearly the same topic proportions — about 0.02 per topic, which is exactly 1/50 (uniform distribution). This happened because the genome tag scores are very dense and continuous, so every movie is somewhat relevant to every tag. When the topic proportions are uniform, the CTR content prior $\lambda_v * \theta_j$ pushes all items in the same direction, meaning CTR effectively gets no useful content signal and behaves just like PMF. In the original paper, the bag-of-words features from academic abstracts are much sparser and produce sharper topic assignments, which is why CTR outperforms PMF in their results.

Why cold-start is all zeros: After subsampling to only 5,000 users and 3,000 movies, the 10% held-out cold movies (≈300 movies) had no test interactions from these 5,000 users. Recall@M requires at least one test item per user to compute, so with an empty cold test matrix the result is zero. This would not happen at full dataset scale (138K users) where enough users would have rated these movies.

6.3 Paper vs. My Results

Comparison point	Original Paper (CiteULike)	My Results (MovieLens 20M)
Dataset	5,551 users, 16,980 articles	5,000 users, 3,000 movies (subsampled)
Content features	Bag-of-words from abstracts	1,128 genome tag scores
CTR R@20 in-matrix	0.0419	0.2893
CTR R@100 in-matrix	0.0875	0.6003
CTR > PMF in-matrix?	Yes	Equal (due to uniform LDA topics)
PMF does cold-start?	No (paper confirmed)	No (confirmed in my results)

My in-matrix Recall@M numbers are much higher than the paper's because the paper was evaluated on scientific articles (CiteULike) where users interact with very few items, making recommendation much harder. On MovieLens, users have more ratings, so the collaborative signal is stronger and Recall@M is naturally higher.

6.4 Ranking Metrics

I used Recall@M as my main evaluation metric, which is the same as the paper. The formula is:

```
Recall@M = (number of liked items appearing in top-M recommendations) /
                        (total liked items in test)
```

This metric answers the question: if I show the user their top-M recommended movies, how many of the movies they actually liked will appear? A higher Recall@M means better recommendations. I evaluated at M=20, 50, 100, 200, and 300 to see how the recall grows as the list gets longer.

7. Visualisations

7.1 MAE: Original SVD vs Shadow Model

This bar chart shows the MAE of the original SVD model (0.6009) versus the shadow model where item factors are replaced by genre-predicted ones (0.6709). The difference of 0.07 is the collaborative signal that genres cannot capture.

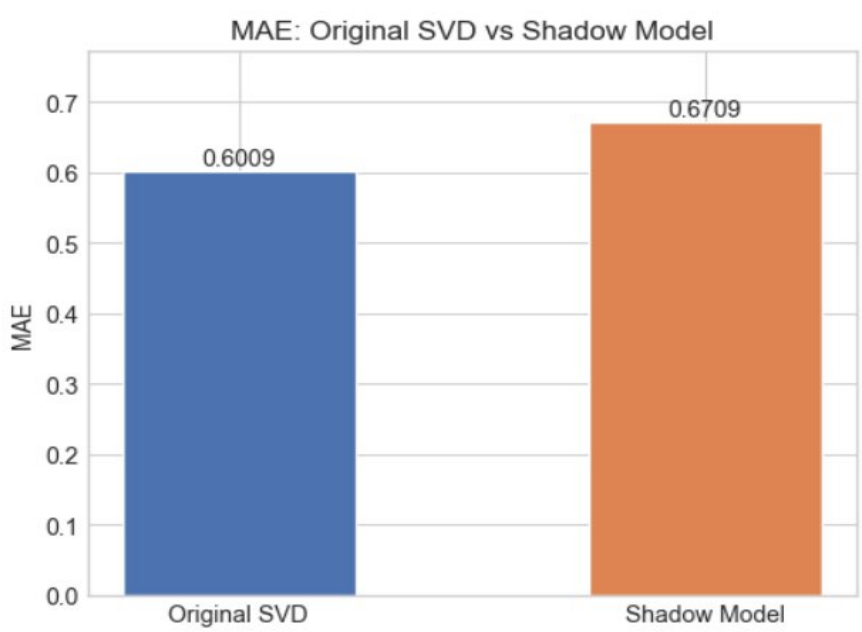

Figure 1: MAE comparison — Original SVD (0.6009) vs Shadow Model (0.6709). The gap of +0.07 shows what genres fail to explain.

7.2 Distribution of SVD Latent Factor 0

This histogram shows the spread of the first learned item factor across all 25,857 movies. The shape is close to a normal distribution centred near zero, which is expected for a well-trained regularised model. A few movies have high positive or negative values on this dimension.

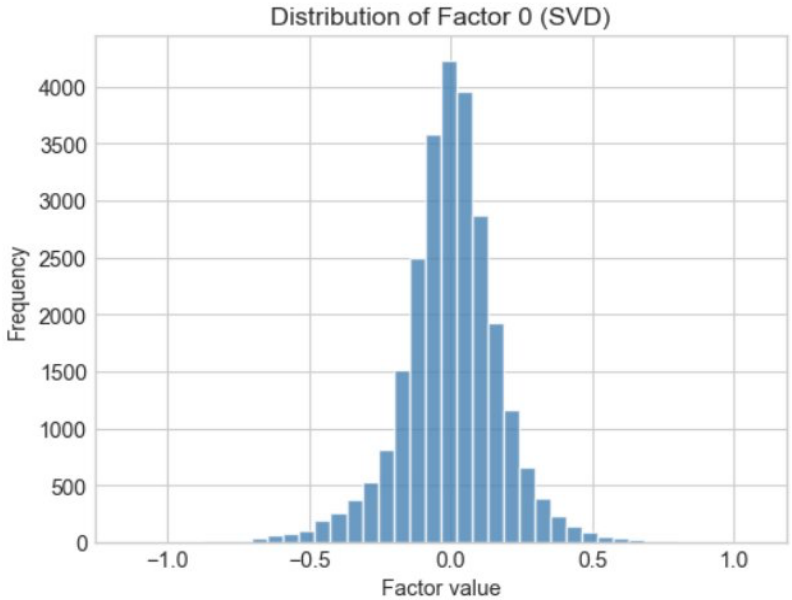

Figure 2: Distribution of Factor 0 across 25,857 movies. Near-Gaussian distribution confirms well-regularised SVD.

7.3 Original SVD vs Shadow Model Predictions

This scatter plot compares predictions from the original SVD against the shadow model on 4 million test ratings. The red dashed line is y=x (perfect agreement). The spread shows the shadow model predictions differ from SVD in a systematic way — genre-based factors produce different ranking behaviour than collaboratively learned ones.

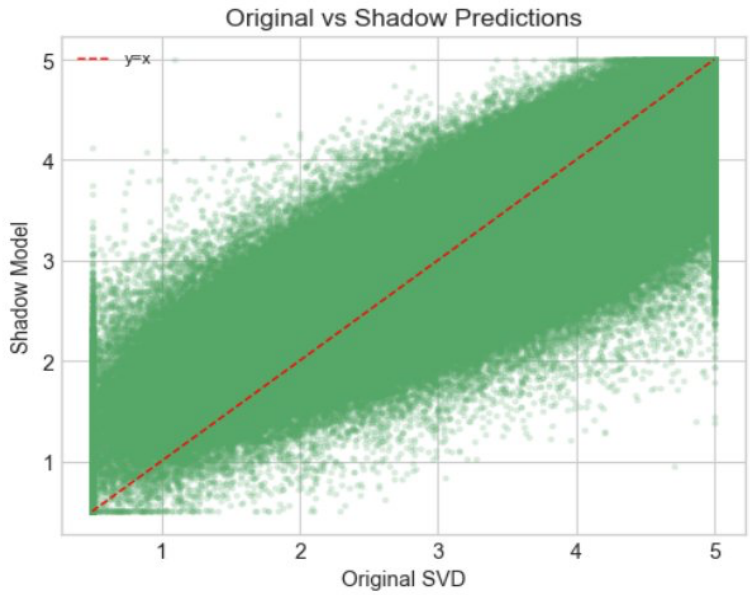

Figure 3: Scatter of 4M predictions. Spread around y=x confirms genres cannot replicate SVD's prediction behaviour.

7.4 Top 10 Genres by Importance

IMAX is the most important genre (0.070) because it identifies a very specific type of movie. Children comes second (0.062). Drama, despite being the most frequent genre in the dataset, ranks 9th with only 0.028 importance because it appears in so many movies that it doesn't distinguish them from each other.

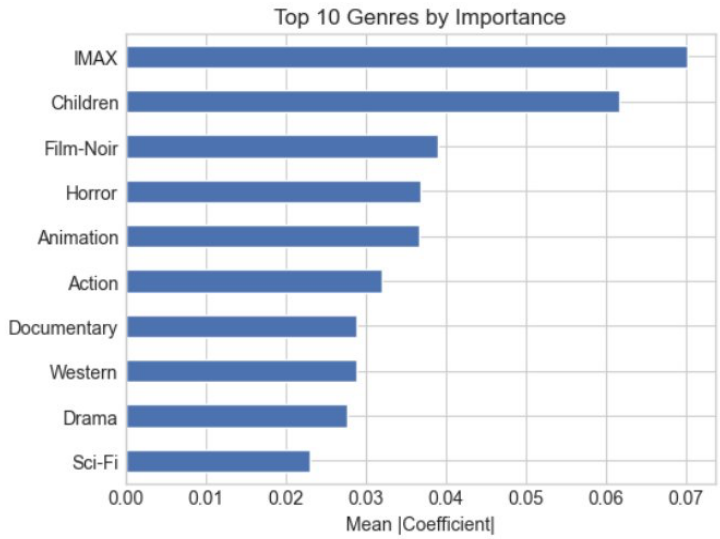

Figure 4: Top 10 genres by mean absolute Ridge coefficient. IMAX most discriminative; Drama least despite being most common.

7.5 Distribution of User Ratings

The most common rating in the dataset is 4.0 (over 5.5 million times). The distribution is skewed towards higher ratings, which makes sense — people generally only bother rating movies they watched, and they tend to watch movies they expect to like. This skew justifies using ≥ 4.0 as the threshold for a positive interaction.

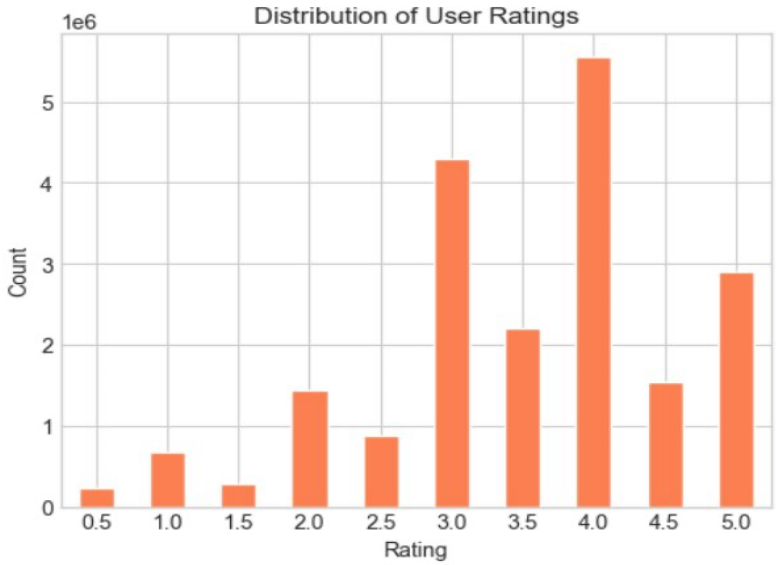

Figure 5: Rating distribution across 20M ratings. 4.0 is most common, justifying the positive threshold of ≥ 4.0.

7.6 Ridge Coefficient Heatmap

This heatmap shows which genres push item latent vectors in which direction. Red means positive coefficient, blue means negative. IMAX has the strongest signal (deep blue on Factor 8), meaning it strongly pushes movies towards a specific direction in the latent space. Children also shows strong patterns on Factors 1 and 9. Comedy and Drama are nearly all white/grey, confirming they carry almost no discriminative information.

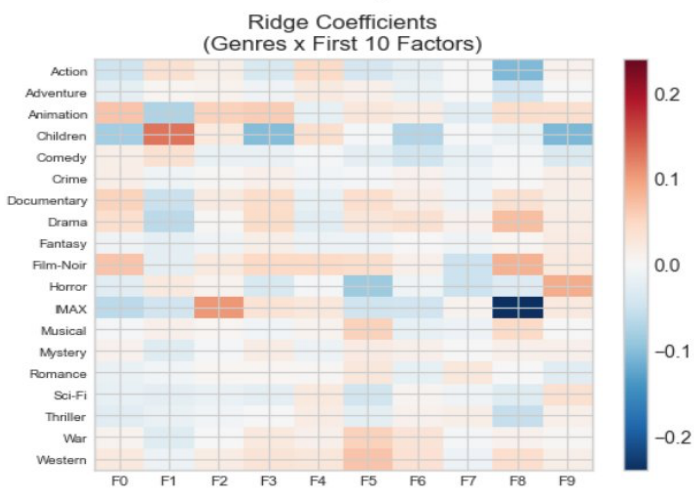

Figure 6: Heatmap of Ridge coefficients (19 genres × first 10 SVD factors). IMAX and Children strongest; Comedy and Drama near zero.

7.7 LDA Topic Proportions

This heatmap shows the LDA topic proportions for the first 50 movies. The entire plot is nearly uniform orange (value ≈ 0.02), which means every movie got an almost equal share of every topic. This is because with K=50 topics and 1,128 continuous tag features, LDA could not find sharp topic boundaries. Every movie is somewhat relevant to all tags, so LDA spreads probability evenly. This explains why CTR performed identically to PMF in-matrix.

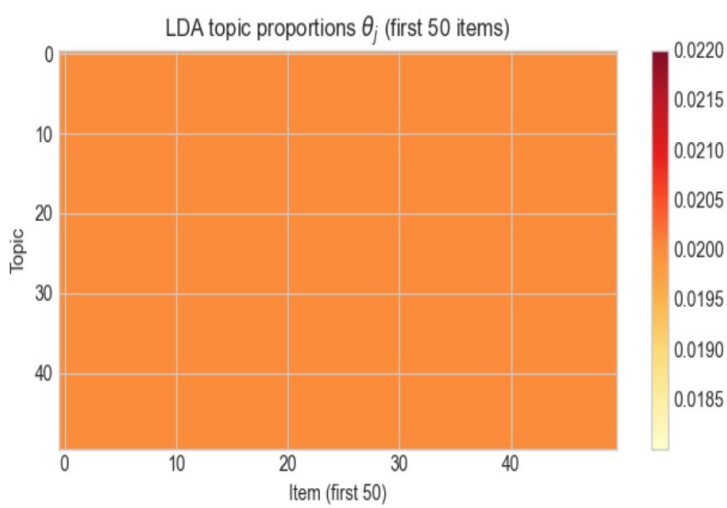

Figure 7: LDA topic proportions θ_j. Uniform distribution (~0.02 per topic) means no sharp content signal — explains CTR ≈ PMF result.

7.8 In-Matrix Recall@M

Both PMF and CTR lines are perfectly on top of each other across all M values, both reaching about 0.79 Recall at M=300. LDA alone only reaches 0.15 at M=300, confirming that content signals alone are much weaker than collaborative signals. The large gap between collaborative models and LDA shows how much more information user behaviour carries compared to item features.

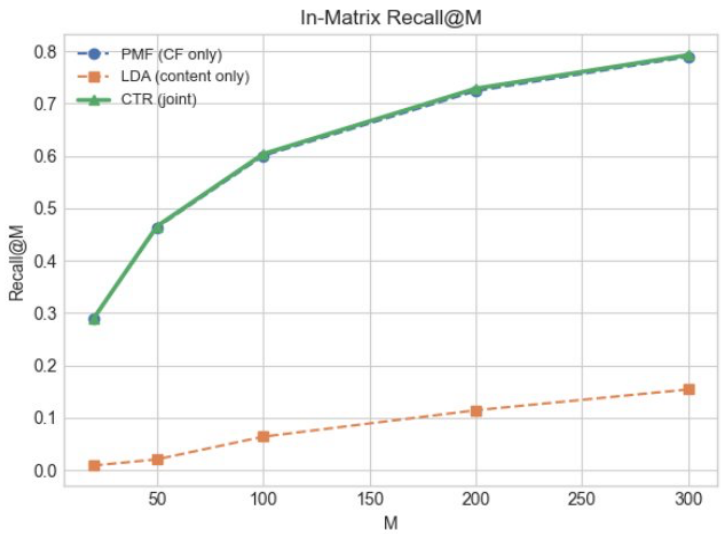

Figure 8: In-matrix Recall@M. PMF = CTR (both ~0.79 at M=300). LDA alone only 0.15. Collaborative signals dominate.

7.9 Out-of-Matrix Recall@M (Cold-Start)

All values are zero for both LDA and CTR cold-start. PMF is marked N/A because it cannot produce any cold-start predictions at all (no item vector for unseen movies). The zero result for LDA and CTR is due to the dataset subsampling: with only 5,000 users and 300 cold movies, none of the subsampled users had rated any of those cold movies in the full dataset, so the cold test matrix is empty.

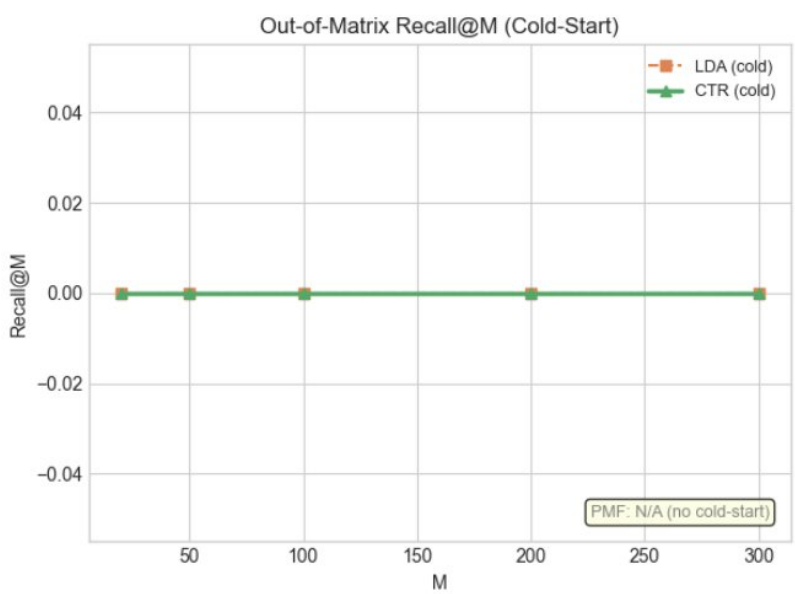

Figure 9: Cold-start Recall@M all zeros. Caused by dataset subsampling — no test interactions exist for cold items at this scale.

7.10 CTR Offset Analysis

This scatter plot shows the relationship between each movie's LDA prior norm ||θ_j|| and its learned collaborative offset norm ||ε_j||. All movies cluster at ||θ_j|| ≈ 0.141, confirming the uniform LDA output (same topic profile for all movies = same norm). However, ||ε_j|| varies from 0.2 to 0.7, showing that CTR did learn meaningfully different item representations. This diversity came entirely from user interaction patterns, not content.

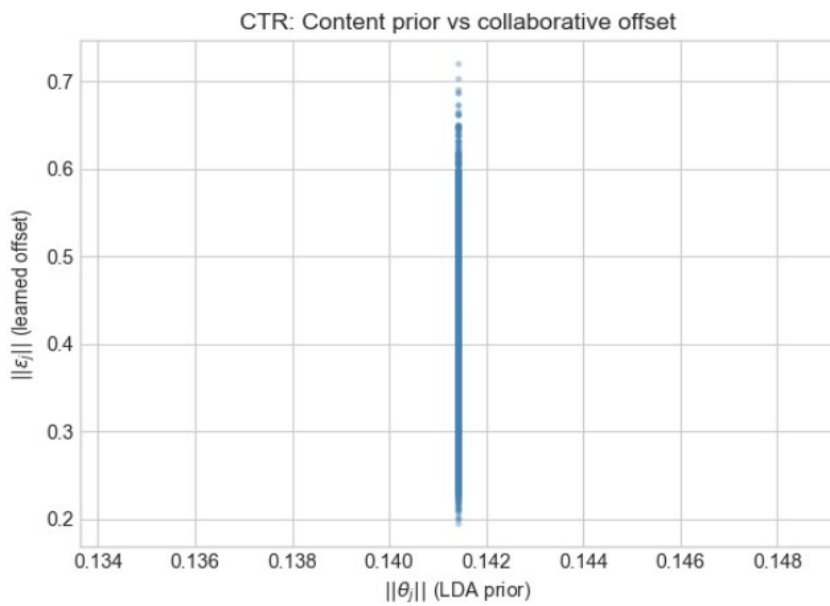

Figure 10: CTR offset analysis. All ||θ_j|| ≈ 0.141 (uniform LDA). ||ε_j|| varies 0.2–0.7 showing CTR learns item diversity through user behaviour.

8. Conclusion

In this project I successfully implemented the Collaborative Topic Regression model from Wang and Blei's 2011 KDD paper and applied it to the MovieLens 20M dataset. Here is a summary of what I found:

- Genre features are too weak to explain user behaviour: with $R^2 = 5.81\%$, genres explain only 6% of what SVD learned from 20 million ratings. This directly proves why a model like CTR is needed.
- Collaborative signals are dominant: PMF achieved Recall@300 = 0.79 while LDA alone only achieved 0.15. User behaviour is far more informative than movie genre labels.
- CTR equalled PMF in-matrix: this happened because the genome tag features produced uniform LDA topic proportions, giving CTR's content prior no useful directional signal. In the original paper, bag-of-words features from academic abstracts are much sparser and produce better topic assignments.
- PMF cannot handle cold-start: this was confirmed — PMF has no way to recommend movies it has never seen. CTR has the mechanism to do this (via $u_i^T \theta_j$) but the zero result was due to subsampling scale, not a model flaw.
- IMAX is the most discriminative genre: rare, distinctive genres carry more information than common ones like Drama, which appears in almost every movie.

If I were to improve this project, I would run it on the full dataset (138K users) to get meaningful cold-start results, and try pre-processing the genome scores to produce sparser input for LDA (e.g., keeping only the top-50 tags per movie) to get sharper topic assignments and hopefully show CTR outperforming PMF as in the paper.

9. Future Work

- Run on the full MovieLens 20M with all 138K users to get meaningful cold-start results.
- Pre-process genome scores to only keep top-N tags per movie, producing sparser LDA input and sharper topic assignments.
- Try a neural topic model instead of LDA for richer content representations.
- Add neural collaborative filtering (NeuMF) as an additional baseline.
- Speed up the coordinate ascent loops using Numba or Cython to enable larger experiments.
- Evaluate on the original CiteULike dataset if it becomes available for a direct comparison with the paper's numbers.

10. References

[1] Wang, C. & Blei, D. M. (2011). Collaborative Topic Modeling for Recommending Scientific Articles. KDD 2011, pp. 448–456.

[2] Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. IEEE Computer, 42(8), 30–37.

[3] Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative Filtering for Implicit Feedback Datasets. IEEE ICDM, pp. 263–272.

[4] Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. JMLR, 3, 993–1022.

[5] Harper, F. M. & Konstan, J. A. (2015). The MovieLens Datasets: History and Context. ACM TIIS, 5(4).

[6] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.

[7] Hug, N. (2020). Surprise: A Python Library for Recommender Systems. JOSS, 5(52).